with specialized tools. And while relationship data works well for collaboration and delegation, it doesn't support context-aware access decisions, such as time-of-day restrictions or device posture. Other models better manage those conditions.

Table 6.2 summarizes the most common strengths and limitations of ReBAC in practice.

Table 6.2 Strengths and limitations of relationship-based access control

Strengths	Limitations
Aligns access decisions with real-world relationships	Not well suited to conditional or contextual access (for example, time or device)
Empowers employees to manage access via intuitive UI	Requires robust graph management and path-tracing capabilities
Avoids role explosion by encoding policy in relationship data	Can be difficult to debug in large or complex graphs
Supports fine-grained delegation without central approval	Access logic can be hard to audit without visibility into the full graph
Enables consistent access control across shared, multi-tenant systems	Assumes the relationship model is well designed and maintained
Makes it easy for non-technical staff to be policy authors	May require custom tooling for administration and governance

As the table shows, ReBAC delivers clear advantages in collaborative, relationship-rich environments, offering intuitive delegation, decentralized policy management, and alignment with how teams actually work. But it also demands strong modeling discipline and operational tooling to avoid unintended access or confusion. The key to using ReBAC effectively is knowing when relationships provide the right foundation—and when other models, like ABAC, are better suited to capture dynamic conditions or nuanced business logic. ReBAC is most powerful when it complements, rather than replaces, other approaches in a layered authorization strategy.

6.1.4 Where ReBAC makes sense

ReBAC performs well in environments where access decisions depend on human or organizational relationships. It naturally suits collaborative platforms that involve sharing, delegation, or scoped teamwork, such as document management systems, developer portals, customer support tools,